



PROJECT REPORT



MEDIQUEUE – HOSPITAL APPOINTMENT & QUEUE MANAGEMENT SYSTEM

PROGRAMMING IN JAVA

Course Code: CSE2006

Project Category: VITyarthi "Build Your Own Project"

Academic Term: Fall Semester 2026

SUBMITTED BY

Student Name: Arnav Kumar

Registration Number: 25BAI10304

**Program: B.Tech Computer Science and Engineering
(AI&ML)**

School / Department: SCAI

Institution: VIT Bhopal University, Madhya Pradesh, India

1. Introduction

MediQueue – Hospital Appointment & Queue Management System is a Java-based application developed to organize common hospital management activities.

The system provides a menu-driven Command Line Interface (CLI) through which hospital staff can manage:

- Patients
- Doctors
- Appointments
- Waiting queues
- Medical records
- Basic reports

The project demonstrates practical implementation of Java concepts such as Object-Oriented Programming, collections, exception handling, file handling, input validation, and date/time handling.

2. Problem Statement

Managing patients, doctors, appointments, queues, and medical records manually can be difficult and time-consuming.

Some common problems include:

- Difficulty maintaining patient information
- Tracking doctor availability
- Appointment scheduling conflicts
- Managing waiting patients
- Handling emergency patients
- Maintaining medical records
- Invalid data entry
- Difficulty retaining records after restarting the application

MediQueue provides a centralized Java-based system to organize these activities through separate modules and persistent local storage.

3. Objectives

The main objectives of MediQueue are:

1. To develop a Java-based hospital management application.
2. To manage patient and doctor information.
3. To provide appointment booking and management.
4. To implement priority-based queue management.
5. To maintain patient medical records.
6. To provide basic reports and statistics.
7. To validate user input and handle errors.
8. To store data persistently using file handling.
9. To demonstrate Java OOP and other programming concepts.

4. Requirements

4.1 Functional Requirements

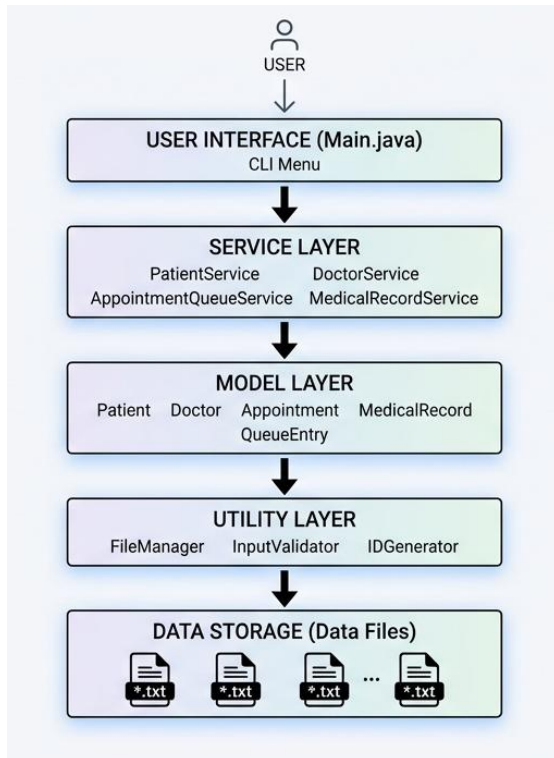
Module	Functions
Patient Management	Register, view, search, update, delete
Doctor Management	Add, view, search, update, delete, availability
Appointment Management	Book, view, search, cancel, reschedule, complete
Queue Management	Add patient, priority handling, call next, view queue
Medical Records	Add, view, search, history, delete
Reports	Appointment, patient, doctor and queue statistics

4.2 Non-Functional Requirements

- Usability: Simple menu-driven CLI.
 - Reliability: Input validation and exception handling.
 - Maintainability: Modular package-based architecture.
 - Performance: Efficient operations for the intended dataset.
 - Scalability: Structure allows future database or GUI integration.
 - Error Handling: Invalid inputs are handled without unexpected termination.
-

5. System Architecture

MediQueue follows a simple layered architecture.



6. System Workflow

Start



Load Existing Data



Display Main Menu



Select Module



Enter Input



Validate Input



Execute Operation



Update Data



Save Data



Display Result



Return to Main Menu



Exit

Appointment Workflow

Patient ID



Validate Patient



Doctor ID



Validate Doctor



Check Availability



Enter Date & Time



Check Conflict



Create Appointment



Save Appointment

Queue Workflow

Patient Arrives



Select Priority



Add to Queue



PriorityQueue

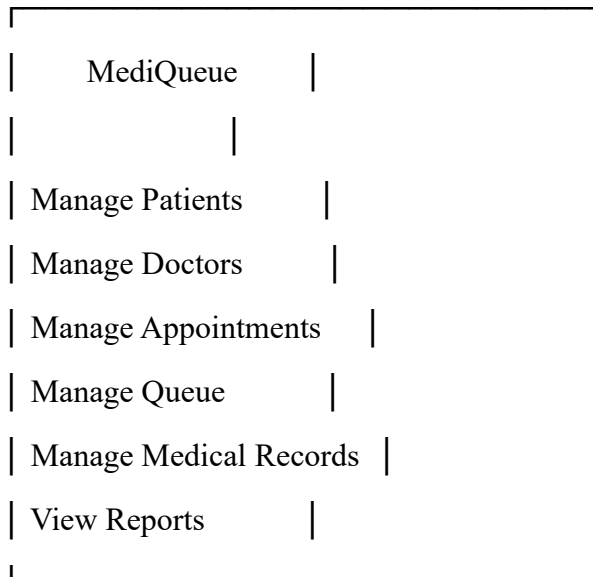


Call Next Patient

7. UML / Design

7.1 Use Case Diagram

Hospital Staff



7.2 Class Structure

User <<abstract>>

/ \

/ \

Patient Doctor

Appointment

MedicalRecord

QueueEntry

Services:

PatientService

DoctorService

AppointmentService

QueueService

MedicalRecordService

Utilities:

FileManager

InputValidator

IDGenerator

8. Implementation

The project is divided into multiple packages:

src/

|— Main.java

|

|— model/

| |— User.java

```
|   |— Patient.java
|   |— Doctor.java
|   |— Appointment.java
|   |— AppointmentStatus.java
|   |— MedicalRecord.java
|   |— QueueEntry.java
|   |— QueueEntryStatus.java
|
|— service/
|   |— PatientService.java
|   |— DoctorService.java
|   |— AppointmentService.java
|   |— QueueService.java
|   |— MedicalRecordService.java
|
|— exception/
|   |— PatientNotFoundException.java
|   |— DoctorNotFoundException.java
|   |— InvalidAppointmentException.java
|   |— QueueException.java
|   |— MedicalRecordException.java
|
|— util/
|   |— FileManager.java
|   |— InputValidator.java
|   |— IDGenerator.java
```

Java Concepts Used

- Classes and Objects
- Encapsulation

- Inheritance
- Abstraction
- Polymorphism
- ArrayList
- PriorityQueue
- HashMap
- Enums
- Exception Handling
- Custom Exceptions
- File I/O
- Regular Expressions
- LocalDate and LocalTime

9. Data Storage

MediQueue uses text files for persistent storage.

data/

|— patients.txt

|— doctors.txt

|— appointments.txt

|— queue.txt

|— medical_records.txt

Example patient record:

P1001|Rahul Sharma|9876543210|rahul@gmail.com|20|Male|O+

Example appointment:

A3001|P1001|D2001|2026-09-20|10:30|SCHEDULED

The application loads existing data when it starts and saves changes to the corresponding files.

10. Screenshots and Results

Add 5–7 important screenshots instead of screenshots for every small operation.

10.1 Main Menu

```
=====
| | | | MEDIQUEUE
| Hospital Appointment & Queue System
=====
1. Patient Management
2. Doctor Management
3. Appointment Management
4. Queue Management
5. Medical Records
6. Reports
7. Exit
=====
Enter your choice:
```

10.2 Patient Management

```
===== PATIENT MANAGEMENT =====
1. Register Patient
2. View All Patients
3. Search Patient
4. Update Patient
5. Delete Patient
6. Back
Enter your choice: █
```

10.3 Doctor Management

```
===== DOCTOR MANAGEMENT =====
1. Add Doctor
2. View All Doctors
3. Search Doctor
4. Search by Specialization
5. Update Doctor
6. Change Availability
7. Delete Doctor
8. Back
Enter your choice: █
```

10.4 Appointment Management

```
=====
APPOINTMENT MANAGEMENT
=====
1. Book Appointment
2. View All Appointments
3. Search Appointment
4. View Patient Appointments
5. View Doctor Appointments
6. Cancel Appointment
7. Reschedule Appointment
8. Complete Appointment
9. Back
=====
Enter your choice: █
```

10.5 Queue Management

```
=====
                        QUEUE MANAGEMENT
=====
1. Add Patient to Queue
2. View Current Queue
3. Search Queue Entry
4. Call Next Patient
5. Remove Patient from Queue
6. Queue Statistics
7. Back
=====
Enter your choice: █
```

10.6 Medical Records

```
=====
                        MEDICAL RECORDS
=====
1. Add Medical Record
2. View All Medical Records
3. Search Medical Record
4. View Patient Medical History
5. View Doctor Records
6. Delete Medical Record
7. Back
=====
Enter your choice: █
```

10.7 Testing Output

```
=====
                        REPORTS & STATISTICS
=====
1. Hospital Dashboard
2. Today's Appointments
3. Appointment Status Report
4. Doctor-wise Appointment Report
5. Patient-wise Appointment Report
6. Queue Report
7. Medical Record Report
8. Back
=====
Enter your choice: █
```

11. Testing

Testing was performed using the dedicated ValidationTest.java class along with manual testing of the major modules.

Test Area	Test
Name	Valid/invalid name
Phone	Valid/invalid phone
Email	Valid/invalid email
Age	Valid/invalid age
Gender	Valid/invalid gender
Blood Group	Valid/invalid blood group
Patient	Add, search, update, delete
Doctor	Add, search, update, delete
Appointment	Book, cancel, reschedule, conflict
Queue	Regular/emergency priority
Medical Records	Add, search, history, delete
Persistence	Reload data after restart

Test Commands

```
javac -d out src/model/*.java src/util/*.java src/exception/*.java src/service/*.java  
src/Main.java
```

```
javac -cp out -d out test/ValidationTest.java
```

```
java -cp out ValidationTest
```

Final Test Result

Total Tests : [ACTUAL NUMBER]

Tests Passed : [ACTUAL NUMBER]

Tests Failed : [ACTUAL NUMBER]

Enter the actual values obtained from your final execution.

12. Challenges Faced

The major challenges during development were:

1. Designing a modular Java architecture.
 2. Managing relationships between patients, doctors, appointments, and records.
 3. Validating patient and doctor IDs.
 4. Preventing appointment conflicts.
 5. Implementing emergency queue priority.
 6. Maintaining persistent data using text files.
 7. Generating unique IDs after application restart.
 8. Handling invalid input and exceptions.
-

13. Learnings

Through this project, we learned:

- Practical implementation of Java OOP.
 - Use of inheritance, abstraction, encapsulation, and polymorphism.
 - Working with Java Collections.
 - Using PriorityQueue for priority-based processing.
 - Implementing custom exceptions.
 - File-based data persistence.
 - Input validation.
 - Modular project organization.
 - Testing Java applications.
 - Using Git and GitHub for version control.
-

14. Future Enhancements

The following features can be added in future versions:

- Database integration using MySQL/PostgreSQL.
- Web-based user interface.

- User authentication.
 - Role-based access control.
 - Appointment notifications.
 - Advanced analytics and dashboards.
 - Digital prescription management.
 - Mobile application.
 - Cloud deployment.
 - Multi-hospital support.
-

15. Conclusion

MediQueue demonstrates the application of Java programming concepts to a practical hospital-management problem.

The system provides modules for patient management, doctor management, appointments, queue management, medical records, and reports.

The project uses a modular architecture along with Java OOP, collections, file handling, validation, exception handling, and date/time APIs.

The application provides a foundation that can be further extended with databases, graphical interfaces, authentication, notifications, and cloud-based services.

16. References

1. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.
2. Java Platform and Java Standard Library Documentation.
3. Java Collections Framework Documentation.
4. Git Documentation.
5. GitHub Documentation.